



Politechnika Łódzka

Instytut Automatyki

Zakład sterowania robotów

TWORZENIE SYSTEMÓW ROBOTYCZNYCH

Temat 7

TF, URDF i Gazebo

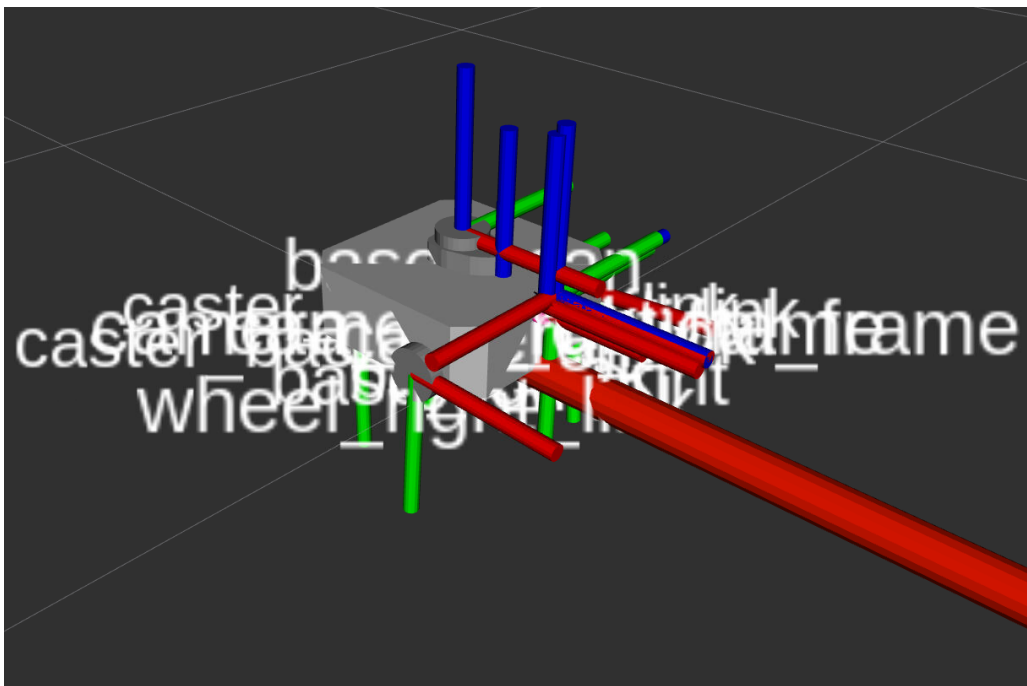
10 marca 2025



Spis treści

1	TF2 – Transformacje układów współrzędnych w ROS2	2
1.1	Publikowanie transformacji w Pythonie	3
1.2	Odczytywanie transformacji w Pythonie	4
1.3	Testy węzłów	5
2	URDF i XACRO	7
2.1	URDF	7
2.2	XACRO	8
3	Gazebo Classic	8
4	Gazebo	9
5	Dokumentacja i inne linki	10

<https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Quaternion-Fundamentals.html>



Zastosowanie TF2 w ROS2:

- Roboty mobilne – śledzenie pozycji platformy względem mapy.
- Manipulatory – określanie pozycji chwytaka względem podstawy robota.
- Systemy wizyjne – analiza położenia kamer i obiektów w otoczeniu robota.
- Fuzja danych sensorycznych – przekształcanie danych z różnych czujników do wspólnego układu odniesienia.

Aby zobaczyć publikowane transformacje w RViz, należy:

1. Uruchomić `ros2 run rviz2 rviz2`.
2. Dodać wtyczkę TF w RViz.
3. Sprawdzić, czy widać TF.

1.1 Publikowanie transformacji w Pythonie

W ROS2 do publikowania transformacji wykorzystuje się bibliotekę `tf2_ros`. Można publikować dwa rodzaje transformacji:

- Transformacja statyczna – określa stały układ odniesienia, który nie zmienia się w czasie. Używana np. do określenia stałej pozycji sensora na robocie.
- Transformacja dynamiczna – zmienia się w czasie, np. pozycja robota względem mapy lub ruch manipulatora.

Transformacje statyczne są publikowane raz i pozostają niezmiennie. Do ich publikowania w Pythonie można użyć `StaticTransformBroadcaster`. W poniższym przykładzie definiujemy układ odniesienia `sensor` względem `base_link` i publikujemy go raz przy starcie.

```
1 import rclpy
2 from rclpy.node import Node
3 from geometry_msgs.msg import TransformStamped
4 from tf2_ros.static_transform_broadcaster import StaticTransformBroadcaster
5
6 class StaticTFPublisher(Node):
7     def __init__(self):
8         super().__init__('tf_static_broadcaster')
9
10        # Utworzenie obiektu umożliwiającego publikowanie statycznych transformacji
11        self.tf_broadcaster = StaticTransformBroadcaster(self)
12
13        # Wywołanie metody odpowiedzialnej za opublikowanie statycznej transformacji
14        self.publish_static_transform()
15
16    def publish_static_transform(self):
17        # Utworzenie i uzupełnienie wiadomości dla transformacji
18        t = TransformStamped()
19        t.header.stamp = self.get_clock().now().to_msg()
20        # Ustawienie układu współrzędnych będący układem odniesienia
21        t.header.frame_id = 'base_link'
22        # Nowy układ współrzędnych
23        t.child_frame_id = 'sensor'
24        # Współrzędne nowego układu wyrażone w układzie odniesienia
25        t.transform.translation.x = 0.5
26        t.transform.translation.y = 0.0
27        t.transform.translation.z = 1.0
28        t.transform.rotation.w = 1.0
29
30        # Opublikowanie transformacji
31        self.tf_broadcaster.sendTransform(t)
32        self.get_logger().info('Opublikowano statyczną transformację.')
33
34    def main():
35        # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
36        rclpy.init()
37        # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy StaticTFPublisher
38        node = StaticTFPublisher()
39
40        # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
41        #   ↪ zakończenie działania programu.
42        rclpy.spin(node)
```

```
43
44     # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
45     node.destroy_node()
46     # Zamyka połączenie z ROS2.
47     rclpy.shutdown()
48
49
50 if __name__ == '__main__':
51     main()
```

Transformacje dynamiczne są używane w sytuacjach, gdzie pozycja i orientacja zmieniają się w czasie, np. podczas ruchu robota. W Pythonie można je publikować za pomocą `TransformBroadcaster`. W tym przypadku `base_link` porusza się po okręgu względem `world`, a transformacja jest publikowana co 0.1 sekundy.

```
1  import rclpy
2  from rclpy.node import Node
3  from geometry_msgs.msg import TransformStamped
4  from tf2_ros import TransformBroadcaster
5  import math
6
7  class DynamicTFPublisher(Node):
8      def __init__(self):
9          super().__init__('tf_static_broadcaster')
10         # Utworzenie obiektu umożliwiającego publikowanie dynamicznych transform
11         self.tf_broadcaster = TransformBroadcaster(self)
12         # Utworzenie timera publikującego nowe wartości transformaty (wywołanie metody broadcast_transform)
13         self.timer = self.create_timer(0.1, self.broadcast_transform)
14         self.angle = 0.0
15
16     def broadcast_transform(self):
17         # Utworzenie i uzupełnienie wiadomości dla transformaty
18         t = TransformStamped()
19         t.header.stamp = self.get_clock().now().to_msg()
20         # Ustawienie układu współrzędnych będący układem odniesienia
21         t.header.frame_id = 'world'
22         # Nowy układ współrzędnych
23         t.child_frame_id = 'base_link'
24         # Współrzędne nowego układu wyrażone w układzie odniesienia
25         t.transform.translation.x = math.cos(self.angle)
26         t.transform.translation.y = math.sin(self.angle)
27         t.transform.translation.z = 0.0
28         t.transform.rotation.w = 1.0
29
30         # Opublikowanie transformaty
31         self.tf_broadcaster.sendTransform(t)
32         self.angle += 0.1
33
34
35 def main():
36     # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
37     rclpy.init()
38     # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy DynamicTFPublisher
39     node = DynamicTFPublisher()
40
41     # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
42     ↪ zakończenie działania programu.
43     rclpy.spin(node)
44
45     # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
46     node.destroy_node()
47     # Zamyka połączenie z ROS2.
48     rclpy.shutdown()
49
50 if __name__ == '__main__':
51     main()
```

1.2 Odczytywanie transformacji w Pythonie

Aby odczytać transformację, można skorzystać z `tf2_ros.Buffer` i `tf2_ros.TransformListener`:

```
1  import rclpy
2  from rclpy.node import Node
3  from tf2_ros import TransformException
4  from tf2_ros import Buffer, TransformListener
5
6  class TFListener(Node):
7      def __init__(self):
8          super().__init__('tf_listener')
9          # Buffer pozwala na przechowywanie historii transformacji, dzięki czemu można uzyskać przeszłe i
10         ↪ przyszłe transformacje.
11         self.tf_buffer = Buffer()
```

```

11     # TransformListener to obiekt nasłuchujący, który odbiera dane o transformacjach publikowane w
    ↪ systemie ROS2. Nasłuchuje transformacje publikowane przez TransformBroadcaster i przechowuje je w
    ↪ tf_buffer.
12     self.tf_listener = TransformListener(self.tf_buffer, self)
13     self.timer = self.create_timer(1.0, self.lookup_transform)
14
15     def lookup_transform(self):
16         try:
17             # pobiera transformację pomiędzy world (ramka nadrzędna) a base_link (ramka podrzędna). Wynik
    ↪ zawiera translację (x, y, z) oraz rotację (quaternion) między podanymi układami
    ↪ współrzędnych.
18             tr = self.tf_buffer.lookup_transform('world', 'base_link', rclpy.time.Time())
19             self.get_logger().info(f"Transformacja: {tr}")
20             self.get_logger().info(f"Wektor translacji: {tr.transform.translation}")
21             self.get_logger().info(f"Przesunięcie x: {tr.transform.translation.x}")
22             self.get_logger().info(f"Wektor rotacji: {tr.transform.rotation}")
23             self.get_logger().info(f"rotacja w: {tr.transform.rotation.w}\n")
24         except TransformException as e:
25             self.get_logger().warn(f"Nie udało się pobrać transformacji: {str(e)}")
26
27
28     def main():
29         # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
30         rclpy.init()
31         # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy TFListener
32         node = TFListener()
33
34         # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
    ↪ zakończenie działania programu.
35         rclpy.spin(node)
36
37         # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
38         node.destroy_node()
39         # Zamyka połączenie z ROS2.
40         rclpy.shutdown()
41
42
43     if __name__ == '__main__':
44         main()

```

1.3 Testy węzłów

Nowe węzły należy dodać do pakietu `example_tsr_project` podobnie jak miało to miejsce z poprzednimi węzłami. Do `setup.py` w miejscu `entry_points` należy dodać:

```

1 entry_points={
2     'console_scripts': [
3         'tf_static_broadcaster = example_tsr_project.tf_static_broadcaster:main',
4         'tf_dynamic_broadcaster = example_tsr_project.tf_dynamic_broadcaster:main',
5         'tf_listener = example_tsr_project.tf_listener:main',
6     ],
7 }

```

Natomiast do `package.xml` należy dodać zależność od pakietu `tf2_ros`:

```

1 <exec_depend>tf2_ros</exec_depend>

```

Następnie ponownie należy przebudować workspace tylko dla pojedynczego pakietu `example_tsr_project`

Po uruchomieniu węzła z dynamicznym broadcasterem

```

1 ros2 run example_tsr_project tf_dynamic_broadcaster

```

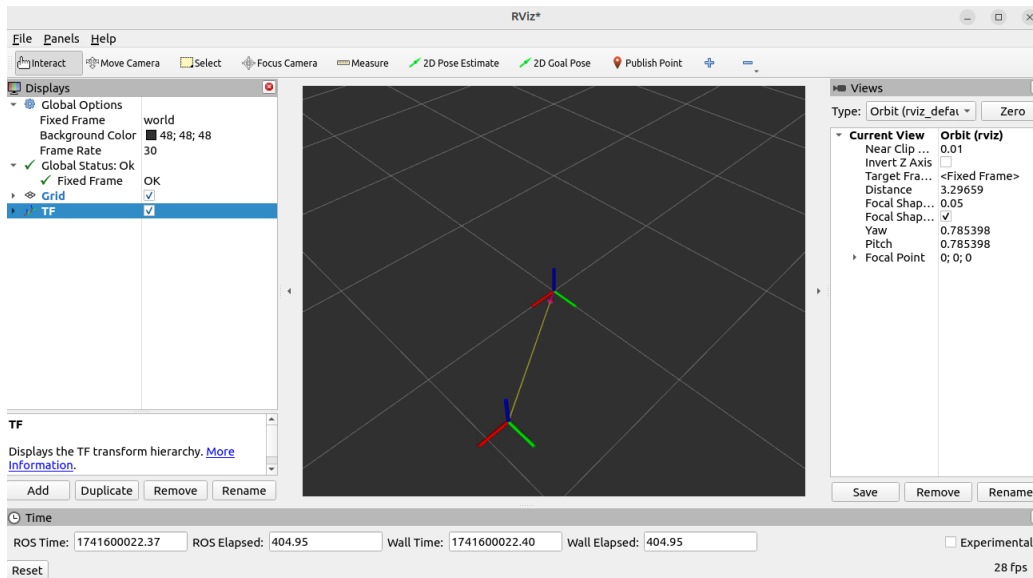
oraz uruchomieniu RViz

```

1 ros2 run rviz2 rviz2

```

wybraniu w Global Options / Fixed Frame wartości `/world` powinny wyświetlić się dwa układy współrzędnych, z których jeden porusza się po okręgu. Opcja z TF powinna być dodana i włączona w RViz:



Rys. 1.2: Podgląd w RViz. Jedna z transformat porusza się po okręgu

Przy uruchomionym `tf_dynamic_broadcaster` można sprawdzić działanie kolejnego węzła odpowiedzialnego za nasłuchiwanie:

```
1 ros2 run example_tsr_project tf_listener
```

W wyniku działania polecenia w terminalu powinno pojawić się:

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 run example_tsr_project tf_listener
[INFO] [1741600505.560464435] [tf_listener]: Transformacja: geometry_msgs.msg.TransformStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.msg.Time(sec=1741600505, nanosec=498935839), frame_id='world'), child_frame_id='base_link', transform=geometry_msgs.msg.Transform(translation=geometry_msgs.msg.Vector3(x=0.9869277063759229, y=-0.1611635889013401, z=0.0), rotation=geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=0.0, w=1.0)))
[INFO] [1741600505.560991981] [tf_listener]: Wektor translacji: geometry_msgs.msg.Vector3(x=0.9869277063759229, y=-0.1611635889013401, z=0.0)
[INFO] [1741600505.561445905] [tf_listener]: Przesunięcie x: 0.9869277063759229
[INFO] [1741600505.561908235] [tf_listener]: Wektor rotacji: geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=0.0, w=1.0)
[INFO] [1741600505.562355305] [tf_listener]: rotacja w: 1.0

[INFO] [1741600506.507267127] [tf_listener]: Transformacja: geometry_msgs.msg.TransformStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.msg.Time(sec=1741600506, nanosec=498881170), frame_id='world'), child_frame_id='base_link', transform=geometry_msgs.msg.Transform(translation=geometry_msgs.msg.Vector3(x=0.668853799347882, y=0.7433939703131195, z=0.0), rotation=geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=0.0, w=1.0)))
[INFO] [1741600506.510160147] [tf_listener]: Wektor translacji: geometry_msgs.msg.Vector3(x=0.668853799347882, y=0.7433939703131195, z=0.0)
[INFO] [1741600506.513411881] [tf_listener]: Przesunięcie x: 0.668853799347882
[INFO] [1741600506.515995409] [tf_listener]: Wektor rotacji: geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=0.0, w=1.0)
[INFO] [1741600506.518979399] [tf_listener]: rotacja w: 1.0

[INFO] [1741600507.507744097] [tf_listener]: Transformacja: geometry_msgs.msg.TransformStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.msg.Time(sec=1741600507, nanosec=499190234), frame_id='world'), child_frame_id='base_link', transform=geometry_msgs.msg.Transform(translation=geometry_msgs.msg.Vector3(x=-0.26416120622352535, y=0.9644785415583553, z=0.0), rotation=geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=0.0, w=1.0)))
[INFO] [1741600507.510366763] [tf_listener]: Wektor translacji: geometry_msgs.msg.Vector3(x=-0.26416120622352535, y=0.9644785415583553, z=0.0)
[INFO] [1741600507.512339533] [tf_listener]: Przesunięcie x: -0.26416120622352535
[INFO] [1741600507.514466725] [tf_listener]: Wektor rotacji: geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=0.0, w=1.0)
[INFO] [1741600507.516541370] [tf_listener]: rotacja w: 1.0

[INFO] [1741600508.505914429] [tf_listener]: Transformacja: geometry_msgs.msg.TransformStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.msg.Time(sec=1741600508, nanosec=498862380), frame_id='world'), child_frame_id='base_link', transform=geometry_msgs.msg.Transform(translation=geometry_msgs.msg.Vector3(x=-0.9543076170347409, y=0.29882598961515106, z=0.0), rotation=geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=0.0, w=1.0)))
[INFO] [1741600508.507945287] [tf_listener]: Wektor translacji: geometry_msgs.msg.Vector3(x=-0.9543076170347409, y=0.29882598961515106, z=0.0)
[INFO] [1741600508.509858744] [tf_listener]: Przesunięcie x: -0.9543076170347409
[INFO] [1741600508.511794070] [tf_listener]: Wektor rotacji: geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=0.0, w=1.0)
[INFO] [1741600508.513684506] [tf_listener]: rotacja w: 1.0
```

Rys. 1.3: Wyświetlanie transformacji pomiędzy world a base_link

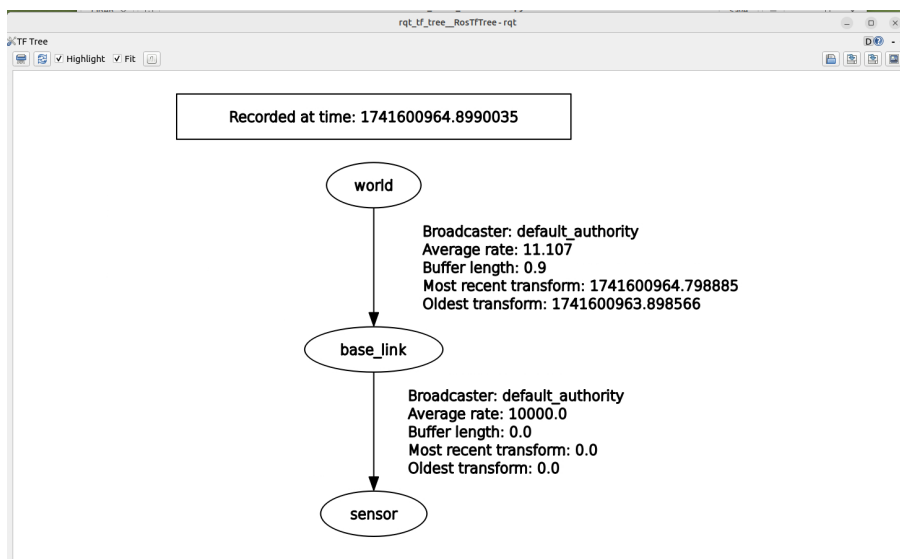
Na koniec uruchomiony zostanie ostatni węzeł od statycznej transformaty:

```
1 ros2 run example_tsr_project tf_static_broadcaster
```

Następnie można podejrzeć drzewo połączeń dla TF2:

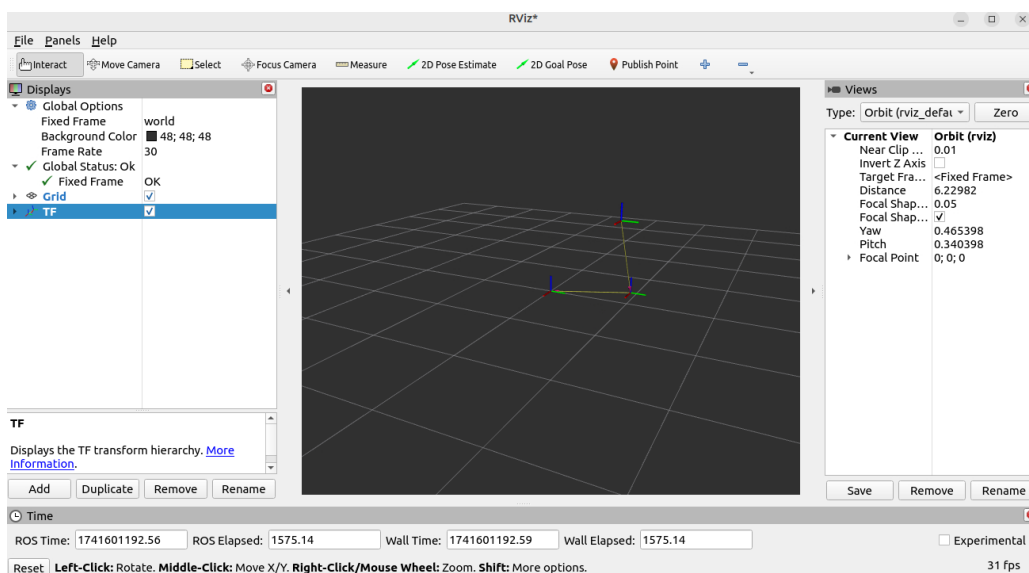
```
1 ros2 run rqt_tf_tree rqt_tf_tree
```

W wyniku działania polecenia pojawia się interfejs graficzny wyświetlający drzewo z ramkami:



Rys. 1.4: Widok drzewa z połączeniami ramek TF

Dodanie statycznej transformaty od `base_link` do `sensor` spowodowało, że w RViz pojawił się dodatkowy układ, który jest na stałe związany z `base_link` i razem z nim się porusza względem układu `world`:



Rys. 1.5: Widok drzewa z połączeniami ramek TF wraz ze statyczną transformatą.

2 URDF i XACRO

2.1 URDF

URDF (Unified Robot Description Format) to format opisu robota w XML, używany w ROS2 do definiowania geometrii, kinematyki i dynamiki systemu. Plik URDF zawiera informacje o:

- Strukturze kinematycznej (np. połączenia między częściami robota),
- Geometrii (kształt i rozmiar elementów robota),
- Właściwościach fizycznych (masa, momenty bezwładności).

Przykład opisu modelu robota **Turtlebot3 Burger** z wykorzystaniem URDF można zobaczyć tutaj:

https://github.com/ROBOTIS-GIT/turtlebot3/blob/main/turtlebot3_description/urdf/turtlebot3_burger.urdf

2.2 XACRO

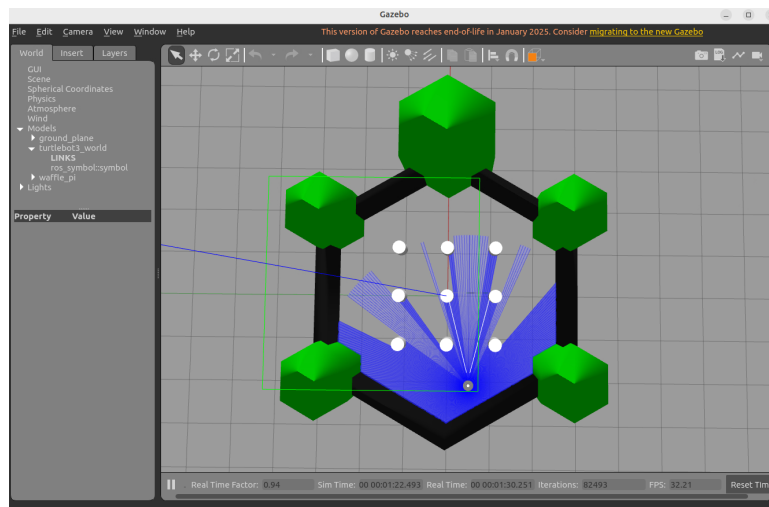
XACRO (XML Macros) to rozszerzenie dla URDF, które pozwala na stosowanie makr, zmiennych i parametrów w plikach URDF. Umożliwia to bardziej elastyczne i czytelne definiowanie modeli robotów poprzez unikanie powielania kodu.

Przykład opisu modelu robota **create** z wykorzystaniem URDF można zobaczyć tutaj:

https://github.com/AutonomyLab/create_robot/blob/humble/create_description/urdf/create_base.urdf.xacro

3 Gazebo Classic

Gazebo Classic to zaawansowany symulator do testowania i rozwijania systemów robotycznych w ROS2. Pozwala na realistyczne odwzorowanie fizyki, interakcji robotów ze środowiskiem oraz pracy czujników w wirtualnym świecie. Jest szeroko stosowany w badaniach, edukacji oraz przemyśle do testowania algorytmów przed wdrożeniem ich na rzeczywiste roboty.



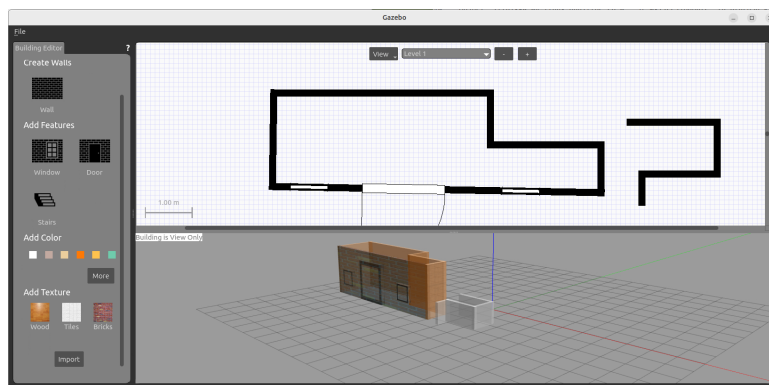
Rys. 3.1: Widok symulowanego robota TurtleBot3 waffle_pi w symulacji

Funkcjonalności Gazebo:

- Zaawansowana fizyka. Gazebo Classic obsługuje różne silniki fizyczne (ODE, Bullet), które symulują:
 - Grawitację i tarcie
 - Zderzenia między obiektami
 - Dynamiczne interakcje robotów z otoczeniem
- Obsługa czujników m.in.:
 - LIDAR-ów
 - Kamer RGB i głębi

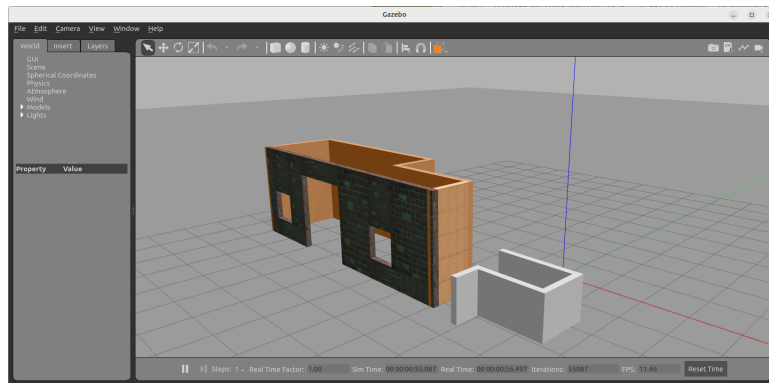
- IMU
- GPS
- Modelowanie robotów z URDF/XACRO
- Możliwość symulacji wielu robotów
- Integracja z ROS2. Możliwość uruchamiania symulacji razem z ROS2 i publikowania danych na tematy ROS2. Symulowane czujniki mogą być analizowane jak rzeczywiste urządzenia.
- Dynamiczne modyfikowanie środowiska. Można dodawać obiekty w trakcie symulacji i zmieniać parametry robotów w czasie rzeczywistym.

Edit/Building Editor umożliwia rysowanie wielopoziomowych budynków w środowisku symulacyjnym poprzez dodawanie konfiguracji ścian, drzwi, okien i pięter. Po narysowaniu modelu można go zapisać. Niestety po jego zapisaniu nie będzie można go później edytować. Przykładowy widok z okna przedstawia:



Rys. 3.2: Widok okna Edit/Building Editor

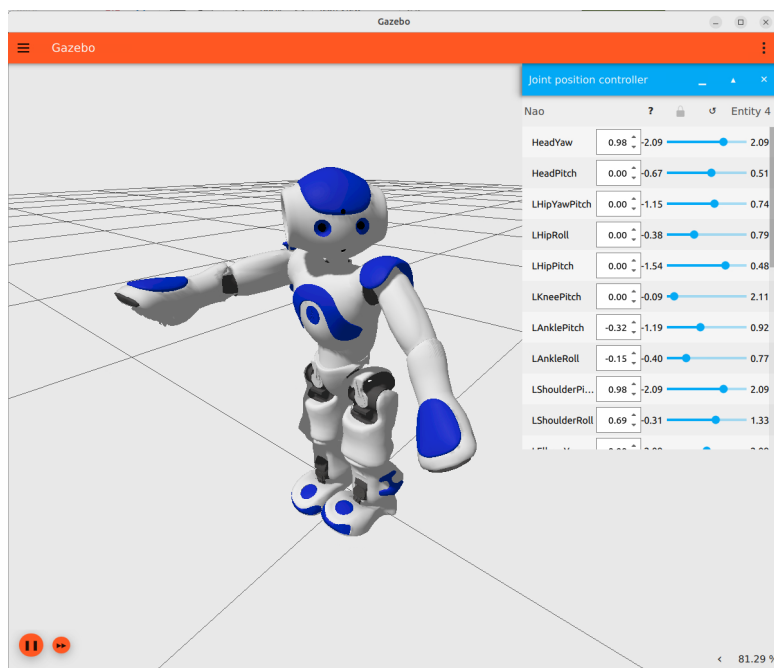
Po zapisaniu modelu świat symulacji wygląda następująco:



Rys. 3.3: Widok świata po zamknięciu edytora do rysowania budynków

4 Gazebo

Gazebo Ignition (obecnie znane jako Gazebo) to nowa wersja popularnego symulatora Gazebo Classic, zaprojektowana z myślą o bardziej modułowej architekturze, lepszej integracji z ROS2 oraz poprawionej wydajności.



Rys. 4.1: Widok symulowanego robota Nao w symulacji

5 Dokumentacja i inne linki

Rozdział może zawierać dokumentację poleceń, linki do źródeł na podstawie, których powstała instrukcja oraz inne linki pozwalające poszerzyć wiedzę.

1. Dokumentacja ROS Humble

<https://docs.ros.org/en/humble/index.html>

2. Więcej o TF

<https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Tf2-Main.html#>

<https://wiki.ros.org/tf2>

3. Więcej o URDF

<https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Building-a-Visual-Robot-Model-with-URDF-from-Scratch.html>

<https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html>

4. Więcej o XACRO

<https://docs.ros.org/en/foxy/Tutorials/Intermediate/URDF/Using-Xacro-to-Clean-Up-a-URDF-File.html>

5. Więcej o symulatorze Gazebo

<https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Gazebo/Simulation-Gazebo.html>

6. Więcej o pluginach i czujnikach w Gazebo

https://docs.ros.org/en/rolling/p/gazebo_plugins/generated/file__tmp_ws_src_gazebo_ros_pkgs_gazebo_plugins

7. Więcej o symulatorze Webots w ROS2

<https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Webots/Simulation-Webots.html#>